

Stock DP Dry Run Diagram

This code solves **LeetCode 338 — Counting Bits**.

The key idea is very simple:

The number of 1s in the binary representation of i = number of 1s in $i \gg 1$ + the last bit of i .

Your entire solution is built around this one observation.

1. What is the question?

For every number from 0 to n , we need to find how many 1s are present in its binary representation.

For example, if:

```
text
n = 5
```

Binary representations:

```
text
0 → 000 → 0 ones
1 → 001 → 1 one
2 → 010 → 1 one
3 → 011 → 2 ones
4 → 100 → 1 one
5 → 101 → 2 ones
```

So:

```
text
ans = [0, 1, 1, 2, 1, 2]
```

2. The important observation

Look at any number i .

Suppose:

```
text
i = 13
```

Binary:

```
text
13 = 1101
```

Now do:

```
C++
i >> 1
```

Right shift by one position:

```
text
1101
↓
0110
```

0110 is 6.

So:

```
text
13 >> 1 = 6
```

Now compare:

```
text
13 = 1101
6 = 0110
```

The rightmost bit of 13 was removed.

Therefore:

```
text
number of 1s in 13
=
number of 1s in 6
+
rightmost bit of 13
```

For 13:

```
text
ones(13)
= ones(6) + 1
= 2 + 1
= 3
```

And indeed:

```
text
1101 → three 1s
```

That's the entire intuition.

3. Understanding this line

```
C++
ans[i] = ans[i >> 1] + (i & 1);
```

Break it into two pieces:

```
C++
ans[i >> 1]
```

and

```
C++
(i & 1)
```

Part 1: `i >> 1`

This removes the last binary bit.

Example:

```
text
i = 10

10 = 1010

10 >> 1

1010
↓
0101

0101 = 5
```

Therefore:

```
C++
ans[i >> 1]
```

means:

How many 1s were present before the last bit?

4. What does `i & 1` do?

This checks the last bit.

There are only two possibilities.

Case 1: `i` is even

Example:

text

$10 = 1010$

Last bit:

text

0

Therefore:

C++

$10 \& 1 = 0$

Case 2: i is odd

Example:

text

$11 = 1011$

Last bit:

text

1

Therefore:

C++

$11 \& 1 = 1$

So:

C++

$i \& 1$

simply tells us:

text

Is the last bit 0 or 1 ?

or equivalently:

text

Is i even or odd?

5. Therefore the formula becomes

C++

$\text{ans}[i] = \text{ans}[i \gg 1] + (i \& 1);$

means:

text

$\text{ones}(i)$
 $=$
 $\text{ones}(i \text{ without its last bit})$
 $+$
 last bit

That's why this works.

6. Let's completely dry-run it

Suppose:

text

$n = 5$

We create:

C++

$\text{vector<int> ans}(n + 1);$

So:

```
text
ans = [0, 0, 0, 0, 0, 0]
```

Then:

```
C++
ans[0] = 0;
```

Because:

```
text
0 = 000
```

has zero 1s.

i = 1

Binary:

```
text
1 = 001
```

Calculate:

```
C++
i >> 1
```

```
text
001 >> 1 = 000
```

So:

```
text
i >> 1 = 0
```

And:

```
C++
i & 1
```

```
text
001
001
--
001 = 1
```

Therefore:

```
C++
ans[1] = ans[0] + 1
        = 0 + 1
        = 1
```

So:

```
text
ans = [0, 1, 0, 0, 0, 0]
```

i = 2

Binary:

```
text
2 = 010
```

Right shift:

```
text
010 >> 1 = 001
```

Therefore:

```
text
i >> 1 = 1
```

Last bit:

text

```
010 & 001 = 000
```

Therefore:

text

```
i & 1 = 0
```

Now:

C++

```
ans[2] = ans[1] + 0
        = 1 + 0
        = 1
```

So:

text

```
ans = [0, 1, 1, 0, 0, 0]
```

i = 3

Binary:

text

```
3 = 011
```

Right shift:

text

```
011 >> 1 = 001
```

So:

text

```
i >> 1 = 1
```

Last bit:

text

```
011 & 001 = 001
```

So:

text

```
i & 1 = 1
```

Therefore:

C++

```
ans[3] = ans[1] + 1
        = 1 + 1
        = 2
```

text

```
3 = 011
    ↑↑
    2 ones
```

i = 4

Binary:

text

```
4 = 100
```

Right shift:

text

```
100 >> 1 = 010
```

So:

text

```
i >> 1 = 2
```

Last bit:

```
text
100 & 001 = 000
```

Therefore:

```
text
i & 1 = 0
```

Thus:

```
C++
ans[4] = ans[2] + 0
        = 1 + 0
        = 1
```

i = 5

Binary:

```
text
5 = 101
```

Right shift:

```
text
101 >> 1 = 010
```

So:

```
text
i >> 1 = 2
```

Last bit:

```
text
101 & 001 = 001
```

Therefore:

```
text
i & 1 = 1
```

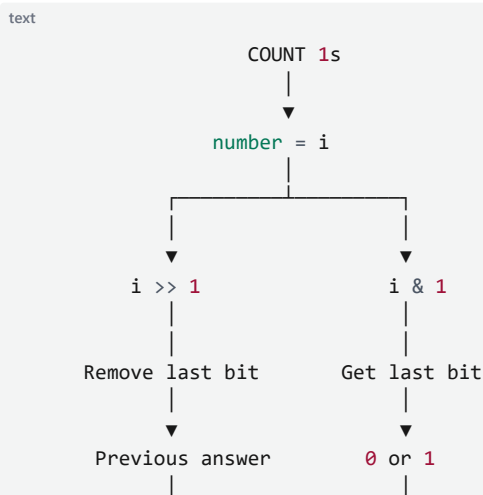
Thus:

```
C++
ans[5] = ans[2] + 1
        = 1 + 1
        = 2
```

Final:

```
text
ans = [0, 1, 1, 2, 1, 2]
```

7. The whole process visually





```
ans[i] = ans[i >> 1]
      +
      (i & 1)
```

8. The most important example: `i = 13`

This makes the trick very clear.

```
text
13 = 1101
```

Split it:

```
text
110 | 1
  ↑   ↑
  |   | last bit
  |   |
  |   | remaining bits
```

Remaining bits:

```
text
110 = 6
```

We already know:

```
text
ans[6] = 2
```

because:

```
text
6 = 110
```

has two 1s.

Last bit:

```
text
1
```

Therefore:

```
text
ans[13]
=
ans[6] + 1

= 2 + 1

= 3
```

Exactly:

```
text
13 = 1101
      ↑
      3 ones
```

9. Why is this DP?

This is essentially **Dynamic Programming** because we don't calculate every number's bits from scratch.

For example, to calculate:

```
text
ans[13]
```

we reuse:

```
text
ans[6]
```

And `ans[6]` was already calculated using:

```
text
ans[3]
```

And:

```
text
ans[3]
```

used:

```
text
ans[1]
```

So there is a dependency chain:

```
text
ans[13]
|
▼
ans[6]
|
▼
ans[3]
|
▼
ans[1]
|
▼
ans[0]
```

And the calculations are:

```
text
ans[0] = 0

ans[1] = ans[0] + 1
       = 1

ans[3] = ans[1] + 1
       = 2

ans[6] = ans[3] + 0
       = 2

ans[13] = ans[6] + 1
        = 3
```

So each state depends on a previously calculated state.

10. Why don't we use normal bit counting?

A straightforward approach would be:

```
C++
for every i:
    count all bits of i
```

For each number, you could repeatedly divide by 2 or shift right.

That takes roughly:

```
text
O(n log n)
```

Your approach does:

```
C++
ans[i] = ans[i >> 1] + (i & 1);
```

which takes $O(1)$ work for each `i`.

Therefore:

```
text
n numbers × O(1)
      ↓
    O(n)
```

Complexity

```
text
Time  = O(n)
Space = O(n)
```

11. One thing you should remember

Don't memorize:

```
C++
ans[i] = ans[i >> 1] + (i & 1);
```

Memorize the binary relationship:

Remove the last bit → count the 1s there → add the last bit.

For example:

```
text
101101
 |
└─ remove last bit
   ↓
 10110
```

Therefore:

```
text
ones(101101)
=
ones(10110)
+
1
```

And for an even number:

```
text
101100
 |
└─ remove last bit
   ↓
 10110
```

```
text
ones(101100)
=
ones(10110)
+
0
```

That's the intuition behind the entire solution.